CS 5100 Final Project: KSP

Camilo Correa Llano

April 12, 2026

1 Introduction

Kerbal Space Program (KSP) is a physics-based spaceflight simulation game in which players assemble spacecraft and launch crews into orbit and beyond [1]. Within the game the player can design and control a spacecraft, which can be launched out of the planet Kerbin to explore the Kerbol system. The physics within KSP are designed to accurately represent orbital and suborbital flight as accurately as possible.

This project attempts to use deep RL, through the Soft Actor-Critic algorithm (SAC), to allow a rocket in KSP to reach orbit, defined as a trajectory whose apoapsis and periapsis are both greater than 80,000 m above sea level (see Fig. 1 for key orbital terminology).

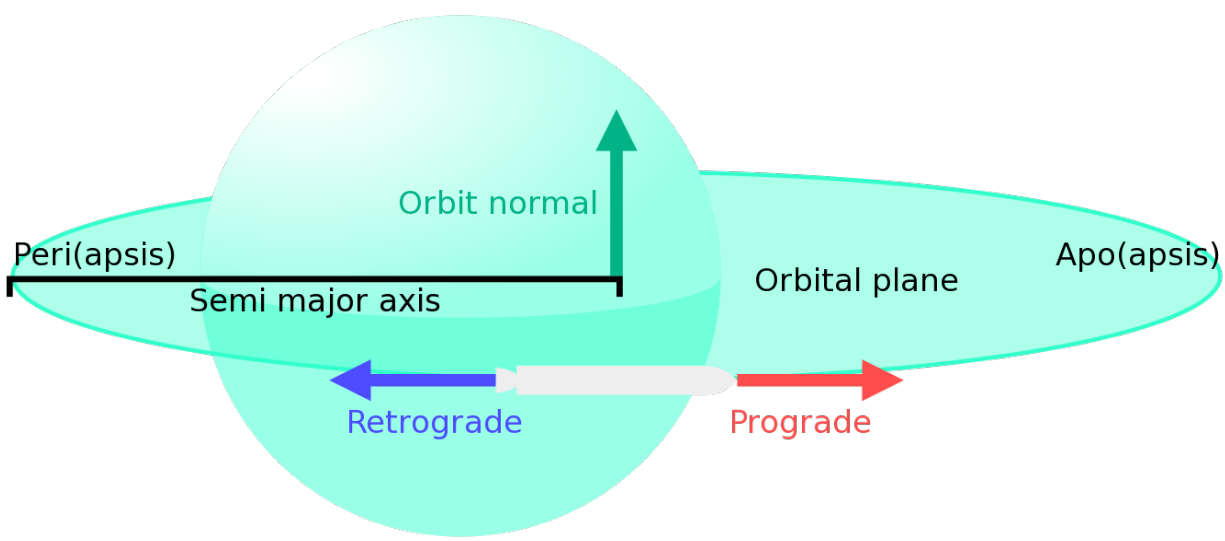

Figure 1: Selected orbital terminology used throughout this report, including apoapsis, periapsis, the orbital plane, and prograde/retrograde directions [2].

Inspiration for this project came from another deep RL project in which an agent was trained to challenge world-record performances in the racing game Trackmania [3]. In addition, the Deep Q-Network (DQN) work of Mnih et al. [5] showed that deep reinforcement learning could succeed in complex game environments, suggesting that related RL methods might also be effective in a simulator as challenging as KSP, though not necessarily DQN itself. This is because DQN operates on a discrete action space, whereas KSP requires a continuous action space. Having worked in robotics before, I instead researched the SAC algorithm [7, 8]; due to its entropy-maximizing objective and continuous action-space support, it seemed like a much better fit for the KSP environment.

There have also been prior attempts to use deep RL to control a vessel in KSP. For example, Jirokoh [4] described an earlier effort that used a much simpler continuous action based on turn angle as a function of altitude. In this project, states were created using the kRPC mod, which exposes a TCP API for connecting to a vessel in KSP. Through that API, information such as altitude, heading, pitch, and orbital telemetry can be gathered directly from the game.

2 Methods

First, as discussed earlier, the emphasis was on choosing an RL algorithm that could operate in a continuous action space. This is because, in KSP, actions such as thrust, pitch, or tilt are not binary but continuous. Thus, the Soft Actor-Critic (SAC) algorithm was chosen, a deep RL algorithm that uses three networks: an actor network and two Q-networks to choose an action distribution given a state. The actor network takes a state and returns, for each action dimension, the mean and standard deviation. The Q-networks take both the state and the action and return the approximate Q-value for the state-action pair. Thus, SAC works by stochastically selecting an action from the actor network and evaluating it using Q-values. These Q-values represent estimates of the total value of the state-action pair. SAC is off-policy and uses training samples from a replay buffer of state, reward, action, and next-state tuples to maximize data efficiency during minibatch backpropagation.

The actor and critic networks were implemented in PyTorch [10]. TorchRL [11] was selected for its accessibility; the environment class acts as a translation layer between TorchRL and the kRPC API. The environment also provides functionality such as quicksave restarts to ensure predictable initial states and vehicle telemetry, including a breakdown of the reward function components.

The environment, `KSPState`, implemented in `ksp_state.py`, is a subclass of `torchrl.envs.EnvBase`. This allows TorchRL to directly interface with the environment. The environment runs on the CPU due to the small tensors involved and game-side events dominating the interaction cost. The observations are exposed as a 10-dimensional vector, including: altitude / target orbit altitude, apoapsis altitude / target orbit altitude, periapsis altitude / target orbit altitude, orbital speed / target orbital speed, vertical speed / target orbital speed, pitch / 90, heading / 180, remaining fuel fraction, current throttle, and min(time to apoapsis/120, 2).

Table 1: Observation vector components ($d = 10$).

Index	**Feature**	**Normalization**
0	Altitude	h/h^*
1	Apoapsis altitude	r_a/h^*
2	Periapsis altitude	r_p/h^*
3	Orbital speed	v/v^*
4	Vertical speed	$\dot{h}/v^*$
5	Pitch	$\theta_p/90$
6	Heading	$\theta_h/180$
7	Fuel fraction	$f \in [0, 1]$
8	Throttle	$u \in [0, 1]$
9	Time to apoapsis	$\min(t_a/120,\ 2)$

All values are normalized to approximately $[-1, 1]$ or $[0, 2]$. Normalizing these values in the approximate region of operation allows the network to converge faster, as no variable is inherently given priority over the others. The target circular orbital speed is

$$v^* = \sqrt{\frac{\mu}{R + h^*}}, \tag{1}$$

where μ is Kerbin's gravitational parameter, R its radius, and h^* the target altitude.

The action space consists of a 3-dimensional vector with components bounded to $[-1, 1]$, representing throttle, pitch, and heading. The policy distribution used is TanhNormal, so outputs naturally lie in $[-1, 1]$.

Reward is shaped using potential-based shaping applied to the clipped periapsis and apoapsis, both clipped to $[0, 1]$. To encourage survival of the craft, a bonus of 0.001 is added for each step survived. To avoid overshooting the orbit target, a -0.01 penalty is applied if the normalized apoapsis exceeds 1.25. If the vessel reaches orbit, a bonus of $+50$ is given; if the vessel is destroyed,

a penalty of -5 is applied. The apoapsis component is scaled by a factor of 10 and the periapsis by 20 within the potential function.

Table 2: Reward function components at each step t.

Component	**Description**	**Value**
Potential shaping	$\Phi(s_t) - \Phi(s_{t-1})$, $\Phi = 10\,\tilde{r}_a + 20\,\tilde{r}_p$	varies
Survival bonus	Reward per step survived	$+0.001$
Overshoot penalty	If $\tilde{r}_a > 1.25$	-0.01
Orbit bonus	On orbit achieved	$+50$
Destruction penalty	Vessel destroyed	-5
Ground penalty	Altitude ≤ 0 at step 10	-100

Potential shaping ensures that the incremental reward for changes in apoapsis and periapsis does not depend on episode length, since SAC optimizes total return. There is one additional penalty: if the altitude is zero or below at step 10, a penalty of -100 is applied to prevent the craft from stalling on the launchpad.

The actor network is an MLP with a 10-dimensional input, two 256-unit ReLU hidden layers, and 2×3 outputs. The outputs are passed through a parameter extractor, yielding the mean (`loc`) and standard deviation (`scale`) for each action dimension. The actor is then wrapped as a probabilistic actor with a TanhNormal distribution, producing bounded continuous actions. The 256-unit hidden layers were selected through experimentation, and ReLU was chosen for its efficiency in deep learning operations.

The critic network, defined as `QVNModel`, concatenates the 10-dimensional observation with the 3-dimensional action to form a 13-dimensional input, then passes it through two 256-unit ReLU hidden layers into a single output unit representing the estimated Q-value. Two critic networks are used within SAC to mitigate Q-value overestimation by taking the minimum of both estimates.

The key hyperparameters are as follows: target orbit altitude (80,000 m), step interval (0.5 s), maximum steps per episode (2,000), hidden size (256), replay buffer capacity (100,000 transitions), batch size (256), number of episodes (500), warmup steps (1,000), updates per environment step (1), actor learning rate (3×10^{-4}), critic learning rate (3×10^{-4}), alpha learning rate (3×10^{-4}), soft-update rate τ (0.005), initial temperature α_0 (0.2), target entropy (`auto`), critic loss (`smooth_l1`), and seed (0).

SAC maximizes the entropy-regularized objective

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t,a_t) + \alpha\,\mathcal{H}(\pi(\cdot \mid s_t))]\,, \tag{2}$$

where $\alpha > 0$ is the temperature and $\mathcal{H}$ is the Shannon entropy.

The critic is trained toward the soft Bellman target

$$y = r + \gamma\Big(\min_{j\in\{1,2\}} Q_{\theta'_j}(s',a') - \alpha \log \pi_\phi(a' \mid s')\Big), \quad a' \sim \pi_\phi(\cdot \mid s'), \tag{3}$$

with target parameters updated as $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$. The critic loss (smooth-ℓ_1 / Huber) is

$$J_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}[L_\delta(Q_\theta(s,a) - y)]\,. \tag{4}$$

Using the reparameterization trick ($a = \tanh(\mu_\phi + \sigma_\phi \odot \varepsilon)$, $\varepsilon \sim \mathcal{N}(0,I)$), the actor is updated by minimizing

$$J_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D},\,\varepsilon\sim\mathcal{N}}\Big[\alpha \log \pi_\phi(a_\phi \mid s) - \min_j Q_{\theta_j}(s,a_\phi)\Big]. \tag{5}$$

The temperature α is adjusted automatically to meet a target entropy $\bar{\mathcal{H}} = -\dim(\mathcal{A})$ via

$$J(\alpha) = \mathbb{E}_{a\sim\pi_\phi}\big[-\alpha\big(\log \pi_\phi(a \mid s) + \bar{\mathcal{H}}\big)\big]\,. \tag{6}$$

For training, a warmup phase of 1,000 steps is used to pre-populate the replay buffer. Thereafter, a standard off-policy loop is followed: collecting environment transitions, sampling minibatches, and performing gradient updates. Exploration is handled by sampling from the stochastic policy rather than acting greedily, replacing the standard epsilon-greedy approach.

The soft target update is performed via the `SoftUpdate` function in the TorchRL library [11]. The replay buffer uses `LazyTensorStorage`, enabling direct sampling to the GPU during training.

A demonstration seeding procedure was implemented to address the cold-start problem. Because early random exploration leads to frequent crashes and KSP is computationally intensive, the `record_demo` function records human-controlled flights as tuples of observation, action, next observation, reward, done, terminated, and truncated. Actions are derived from the current throttle and the changes in pitch and heading between samples. These demonstrations are pre-loaded into the

replay buffer, providing the agent with examples of successful flight trajectories. This data is stored in the `userData` directory.

Checkpointing was implemented to support interrupted training, saving the episode number, total training steps, `loss_module.state_dict()`, optimizer state, and replay buffer, so training can be paused and resumed at any time.

Additionally, comprehensive per-step logging was implemented, recording: `seed, train_step, episode, step_in_episode, eval_flag, success, orbit_time_s, total_return, reward_total, reward_potential, reward_alive, reward_overshoot, reward_orbit_bonus, reward_failure_penalty, reward_ground_penalty, fuel_remaining, fuel_remaining_frac, apoapsis, periapsis, altitude, max_altitude, speed, max_speed, termination_reason, actor_loss, critic_loss, alpha, loss_alpha`. Per-episode summaries include: `seed, train_step, episode, eval_flag, success, orbit_time_s, total_return, fuel_remaining, fuel_remaining_frac, apoapsis, periapsis, max_altitude, max_speed, termination_reason, episode_steps`.

The `main.py` file reconstructs the actor, critic, SAC loss module, and optimizer from the latest checkpoint, then runs the policy in deterministic mode to suppress exploratory behavior during evaluation.

3 Results

The agent was trained for 300 episodes using the SAC algorithm described above. As seen in Fig. 2, the reward for the agent remained relatively consistent throughout training, largely due to the penalty incurred when the vessel is destroyed. During episodes 240 to 250 multiple spikes are seen reaching $\sim$135 total return, corresponding to episodes in which the agent managed to reach orbit. As seen in Fig. 3, episode length mirrors Fig. 2: it increases as the craft gets closer to orbit, nearing the 2,000-step cap, before collapsing near the end of training.

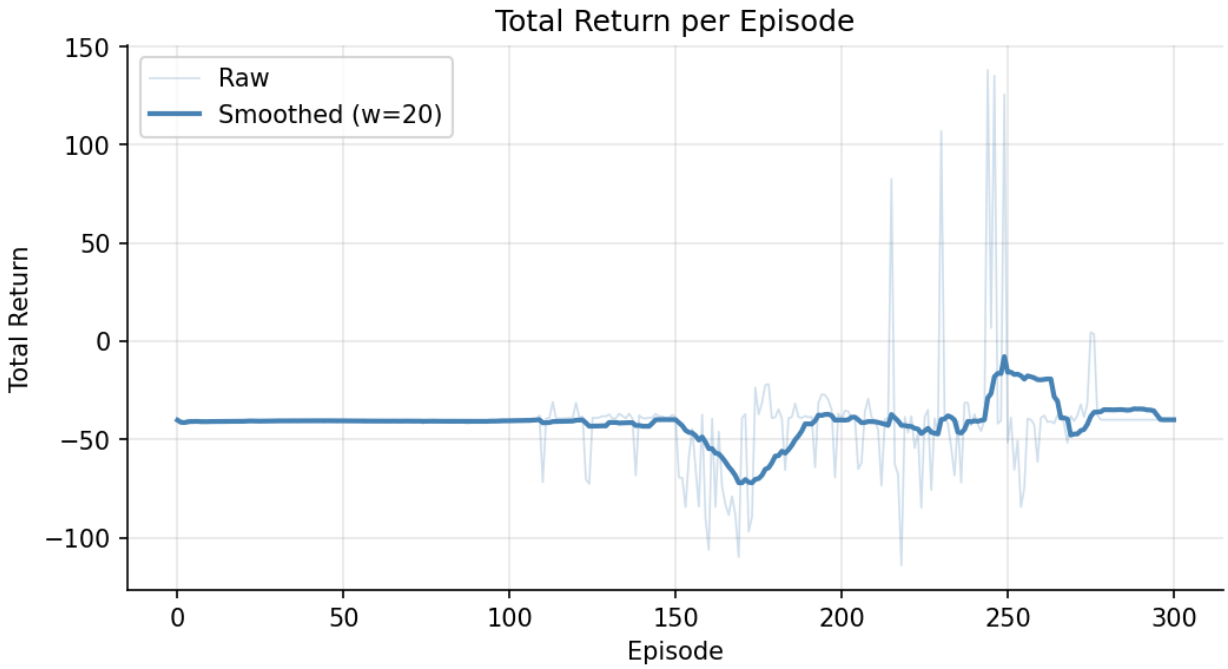

Figure 2: Total return per episode (raw and 20-episode smoothed).

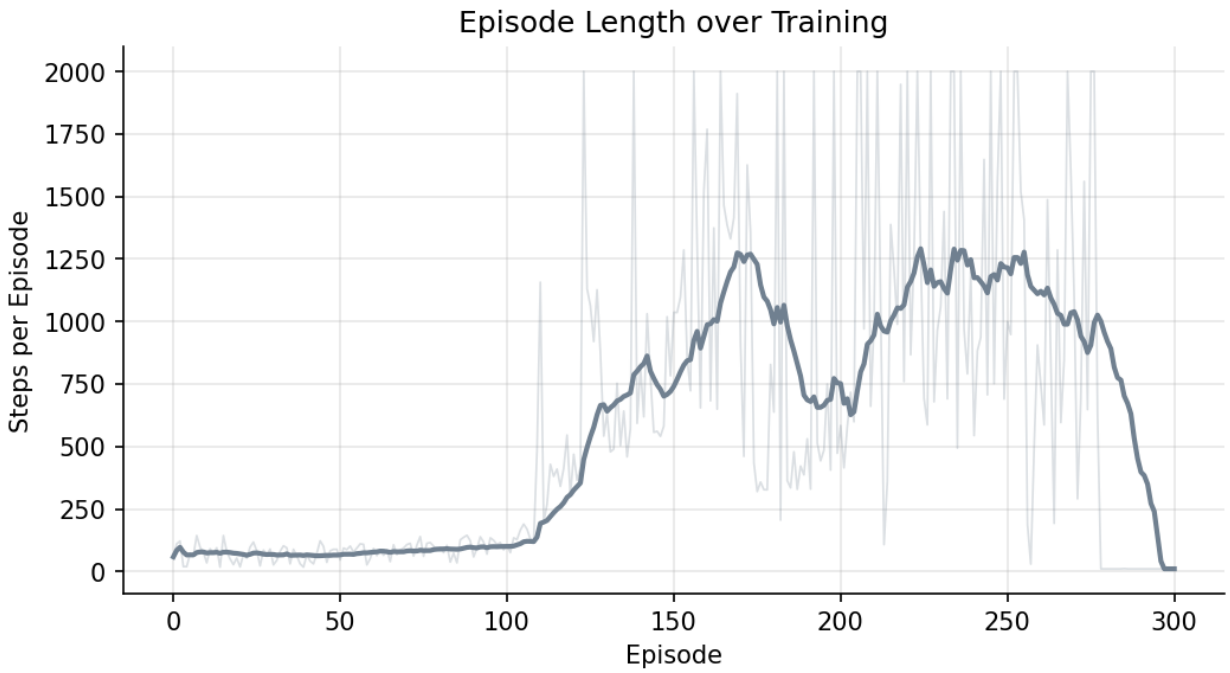

Figure 3: Number of steps per episode over training.

As seen in Fig. 4, the SAC entropy temperature α collapsed severely before the agent could find the orbit reward, meaning the policy became nearly deterministic before encountering the large

positive returns associated with orbit achievement. This likely contributed to the local minimum the model ultimately converged to around episode 270.

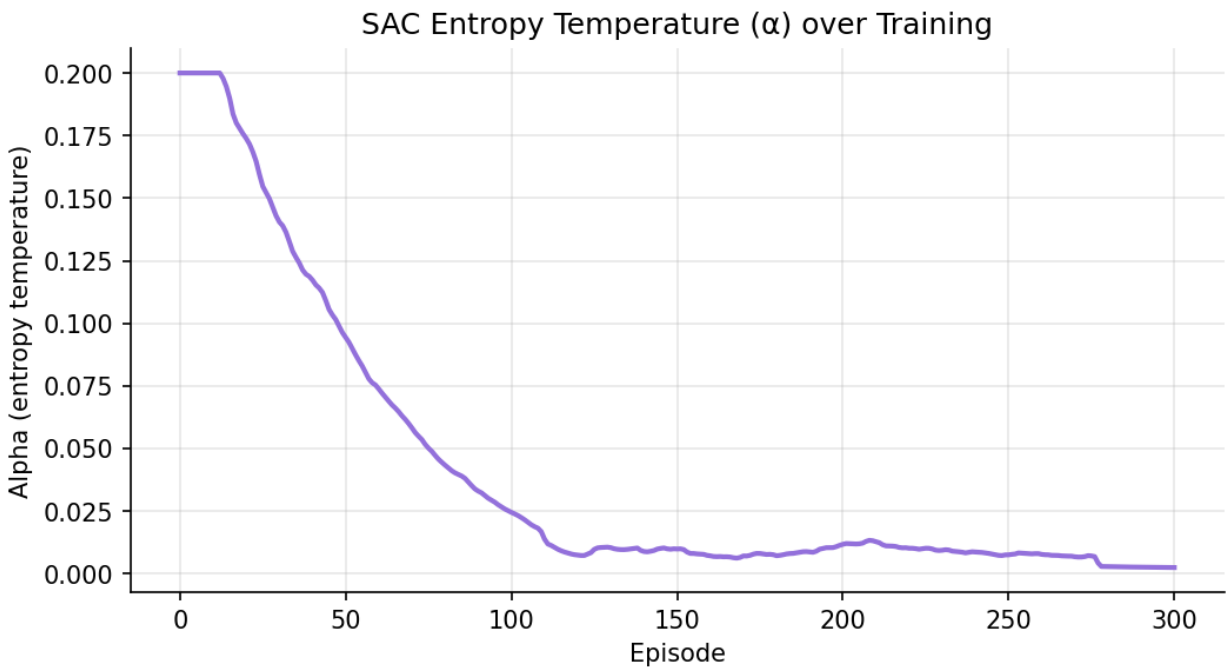

Figure 4: SAC entropy temperature α over training.

This pattern continues in Figs. 5 and 6, where the model reached increasing altitudes approaching the overshoot penalty threshold. Notably, starting around episode 110 the model began reaching space, though mastering the orbital circularization burn proved difficult. The subsequent decrease in apoapsis may also have been partly caused by the overshoot penalty discouraging trajectories that exceeded the target altitude.

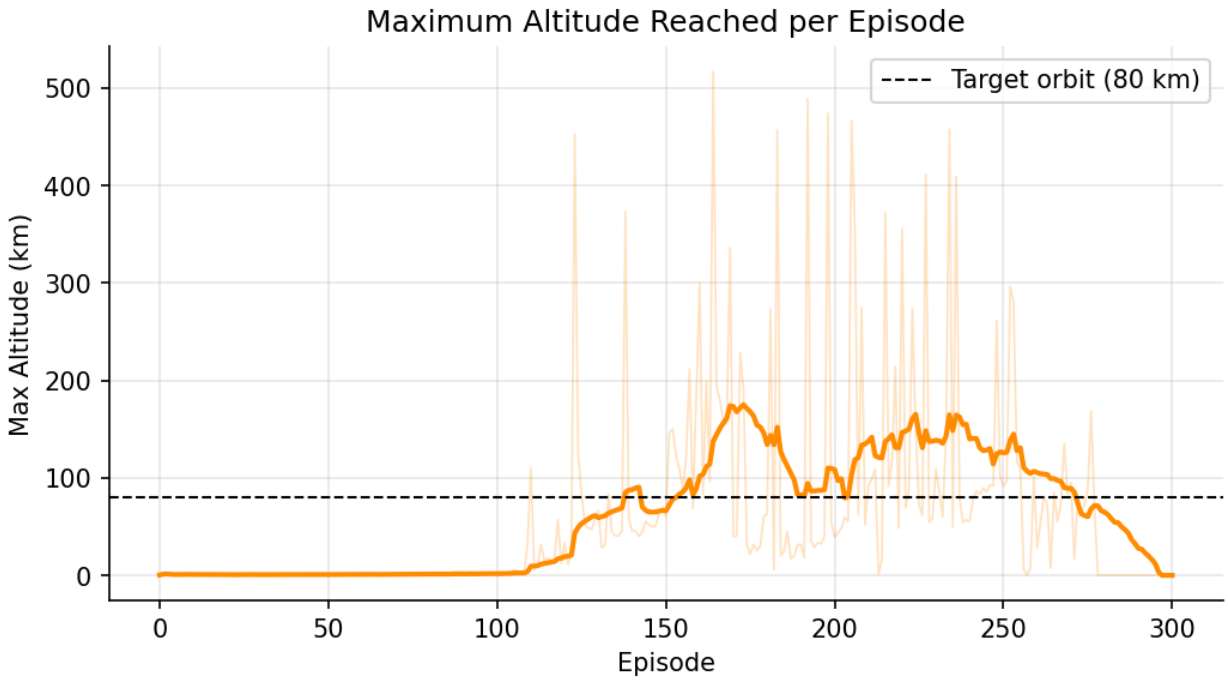

Figure 5: Maximum altitude reached per episode, with the 80 km target orbit indicated.

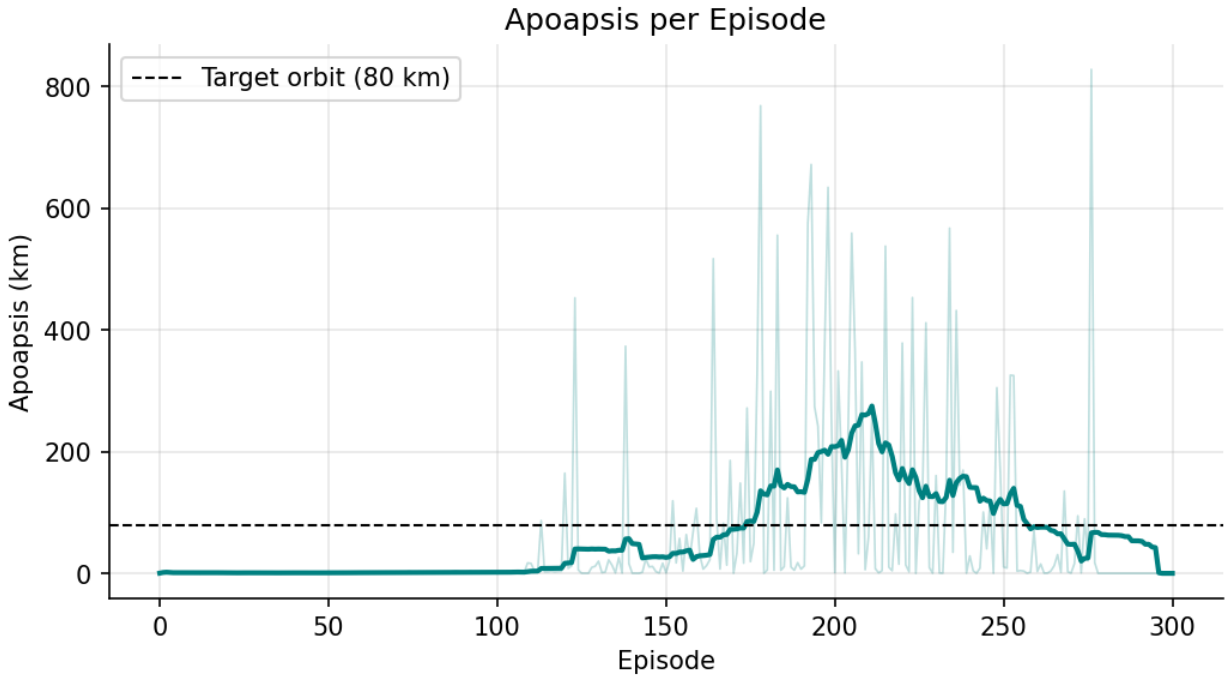

Figure 6: Apoapsis per episode, with the 80 km target orbit indicated.

As seen in Fig. 7, the model did achieve orbit in approximately 20% of episodes within the

20-episode window around episode 250, before collapsing to a local minimum. This is likely because α was too low to significantly influence the Q-value predictions and drive further exploration.

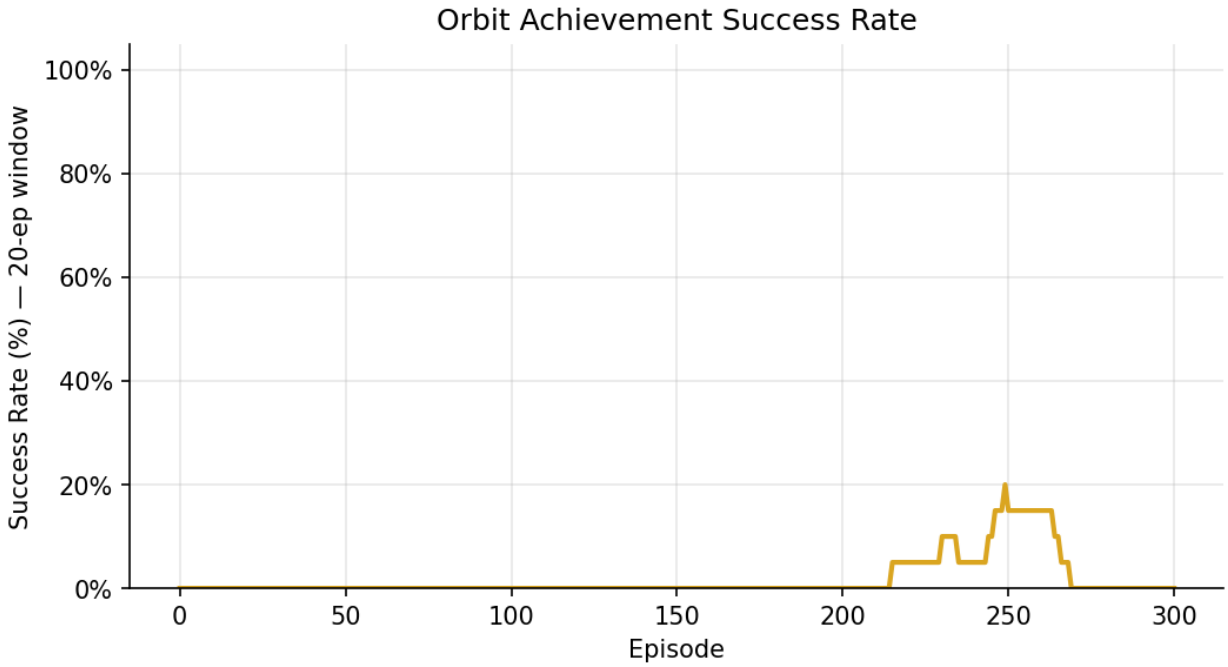

Figure 7: Rolling orbit-achievement success rate over a 20-episode window.

As seen in Fig. 8, most vessels were destroyed (82%), 8% of episodes ended due to the maximum step count being reached, approximately 8% ended with the vessel below ground due to high-speed terrain impacts causing errors in the altitude calculation, and 5 episodes ended in successful orbits.

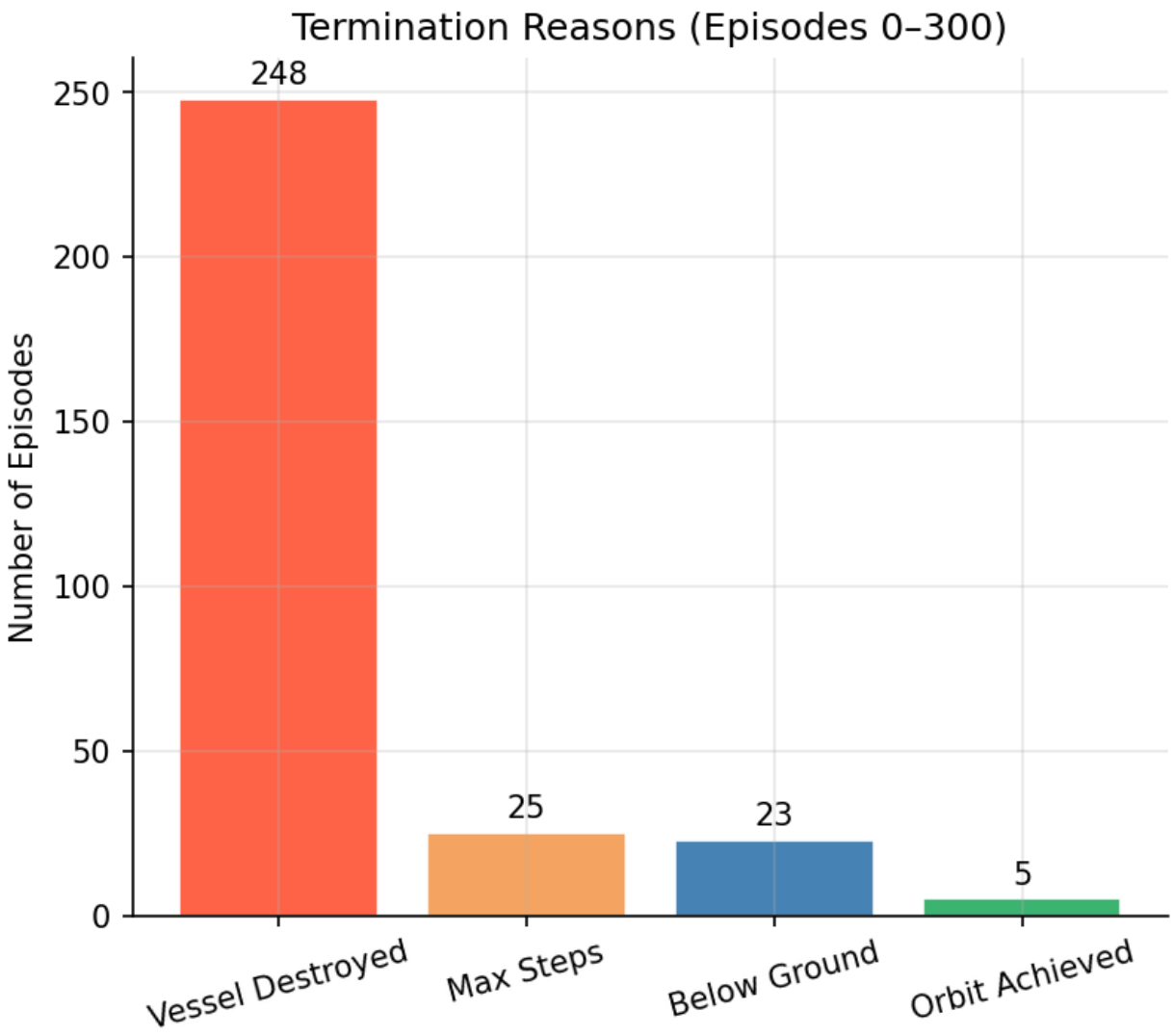

Figure 8: Distribution of episode termination reasons over episodes 0–300.

As seen in Fig. 9, both losses remain near zero until the local minimum of crashing into the ground as quickly as possible is reached. The mean actor and critic losses then suddenly explode toward extreme values, likely as a signal of Q-value overestimation. This is also reflected in Fig. 10, where the reward breakdown becomes primarily dominated by the ground penalty after episode ∼270. Finally, Fig. 11 further illustrates the Q-value explosion, as fuel remaining per episode increases sharply after episode 270, indicating the vessel is being destroyed before expending any fuel.

Figure 9: Mean actor loss and critic loss per episode over training.

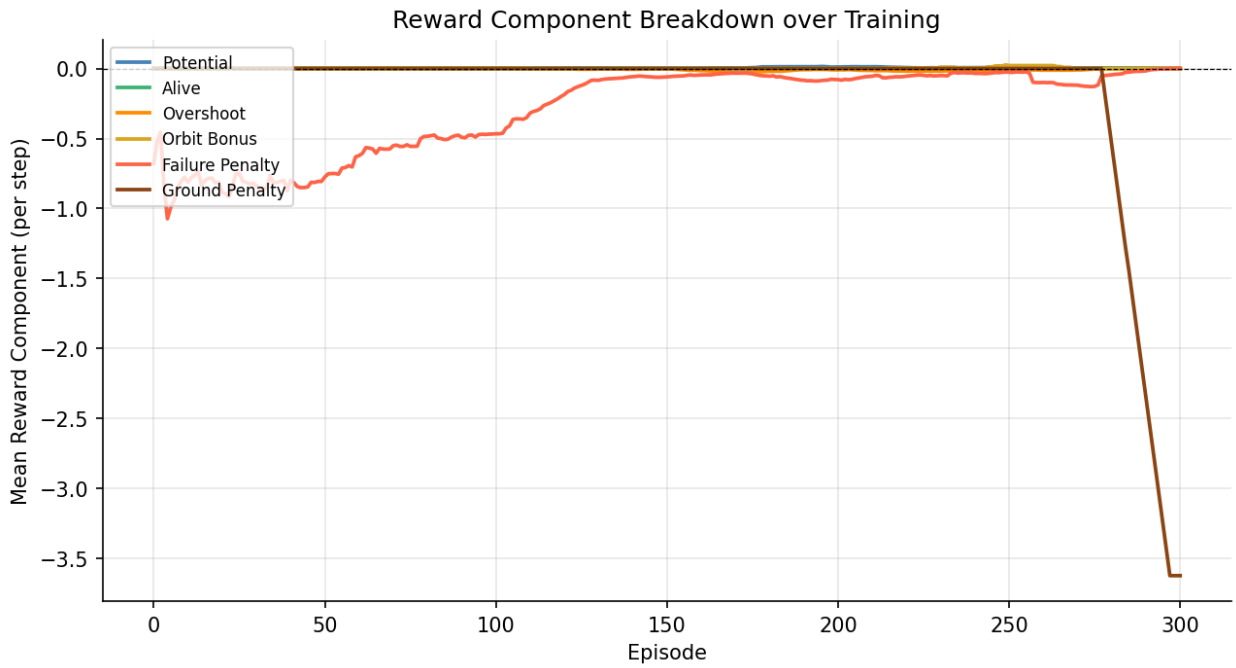

Figure 10: Mean per-step reward components over training.

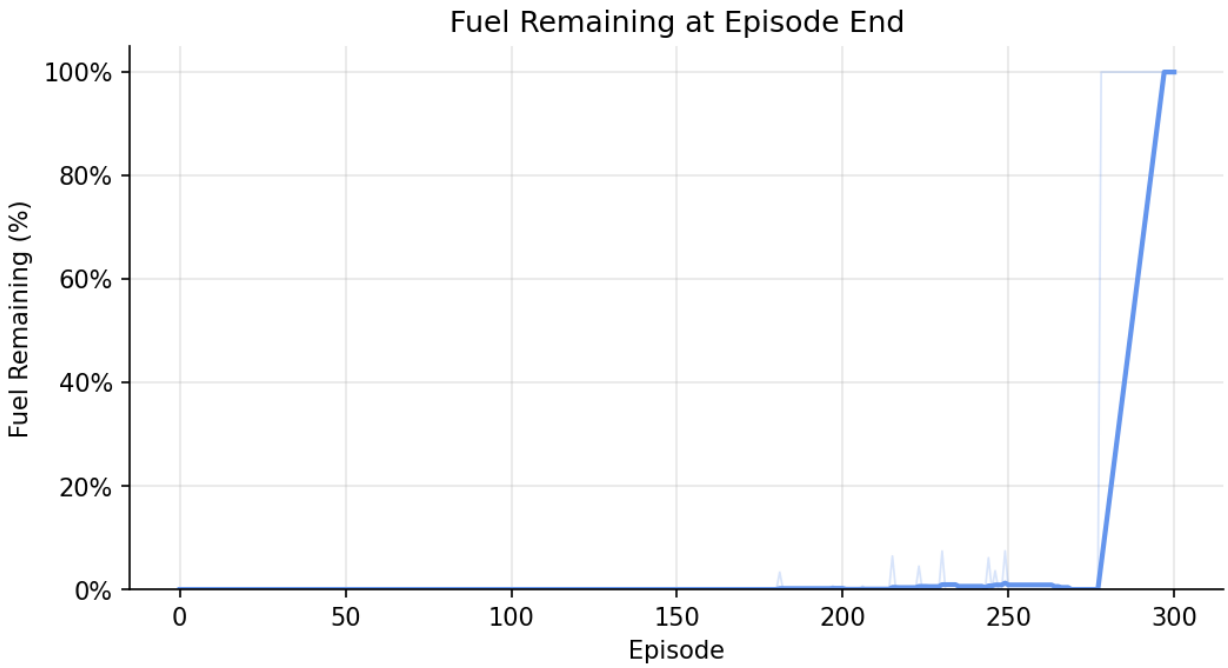

Figure 11: Fuel remaining at episode end over training.

4 Conclusions

The project demonstrated that a Soft Actor-Critic can meaningfully learn complex orbital behavior within Kerbal Space Program, achieving orbit in 5 of the 301 training episodes. Moreover, the agent consistently reached altitudes above Kerbin's atmosphere (70 km) between episodes 130 and 270. The choice of SAC seems justified, as its entropy-maximizing continuous-action approach was better suited to KSP than DQN [5], since thrust regulation and rotation are essential to achieving orbital maneuvers and require a continuous action space.

The primary issue with the approach is training instability [7]. The explosion of the actor and critic losses is likely due to the Q-values of these state-action pairs being severely overestimated, which caused the agent to begin destroying itself as soon as it left the launchpad. Another weakness was the premature decrease of the entropy temperature: by episode 100 α was small enough that high-reward states such as orbit did not significantly alter the policy. One last weakness was that the agent easily learned to reach space but had a very difficult time circularizing the orbit. This is likely because the apoapsis usually exceeded the target, triggering the overshoot penalty, which overshadowed the reward from raising the periapsis.

Compared to previous attempts such as [4], which used continuous actions defined as hand-crafted intervals of turn angles as a function of a single state variable — altitude — the SAC approach presented here is far more generalizable and requires no predetermined action structure. However, that rule-based approach would be far more stable and unlikely to collapse to the local minimum this approach converged toward. A Proportional-Integral-Derivative (PID) controller as used in robotics might reliably reach orbit for a specific rocket, yet would not generalize to other configurations. Lastly, the Trackmania agent [3] shows a similar deep RL approach, though the absence of catastrophic failure states and a vastly different state space make direct comparison difficult.

There are several improvements that could address these weaknesses, such as clipping the critic update and applying layer normalization on the Q-networks to avoid the overestimation issue. Increasing $\bar{\mathcal{H}}$ or using a slower alpha decay schedule would encourage more exploration before committing to a strategy. A third improvement would be a staged reward curriculum: first rewarding apoapsis achievement, then the circularization burn, and finally the orbit bonus. Another direction,

inspired by [9], would be to include a global view of the orbital diagram using convolutional layers in both the Q-networks and actor network. Finally, expanding the number of training episodes would allow more time for the policy to consolidate before instability sets in.

References

[1] Kerbal Space Program, "Kerbal Space Program," [Online]. Available: https://www.kerbalspaceprogram.com/. [Accessed: Apr. 12, 2026].

[2] Kerbal Space Program Wiki, "Terminology," [Online]. Available: https://wiki.kerbalspaceprogram.com/wiki/Terminology. [Accessed: Apr. 15, 2026].

[3] "AI just Broke Trackmania's most Legendary Record," YouTube video. [Online]. Available: https://www.youtube.com/watch?v=zFLQU7OQstY. [Accessed: Apr. 12, 2026].

[4] Jirokoh, "I want to teach a rocket how to fly!," *Kerbal Space Program Forums*, Apr. 3, 2019. [Online]. Available: https://forum.kerbalspaceprogram.com/topic/183388-i-want-to-teach-a-rocket-how-to-fly/. [Accessed: Apr. 12, 2026].

[5] V. Mnih *et al.*, "Playing Atari with Deep Reinforcement Learning," arXiv:1312.5602, Dec. 2013. [Online]. Available: https://arxiv.org/abs/1312.5602. [Accessed: Apr. 12, 2026].

[6] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," arXiv:1511.06581, Nov. 2015. [Online]. Available: https://arxiv.org/abs/1511.06581. [Accessed: Apr. 12, 2026].

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," arXiv:1801.01290, Jan. 2018. [Online]. Available: https://arxiv.org/abs/1801.01290. [Accessed: Apr. 12, 2026].

[8] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft Actor-Critic Algorithms and Applications," arXiv:1812.05905, Dec. 2018. [Online]. Available: https://arxiv.org/abs/1812.05905. [Accessed: Apr. 12, 2026].

[9] C. Hu, K. Yang, Z. Chen, L. Li, and G. Zhao, "Reinforcement Learning with Dual-Observation for General Video Game Playing," arXiv:2011.05622, Nov. 2020. [Online]. Available: https://arxiv.org/abs/2011.05622. [Accessed: Apr. 12, 2026].

[10] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: `https://pytorch.org/`. [Accessed: Apr. 15, 2026].

[11] A. Bou *et al.*, "TorchRL: A data-driven decision-making library for PyTorch," arXiv:2306.00577, 2023. [Online]. Available: `https://docs.pytorch.org/rl/stable/index.html`. [Accessed: Apr. 15, 2026].